

Tracero · 智循

机器人与自驾领域的自动化行为分析专家

一、这个项目在解决什么问题

从一个真实场景说起

一辆自动驾驶仿真车在城市路口行驶，行人从停着的两辆车之间突然冲出——车没刹住，撞了。

或者

一个人形机器人在跳跃的时候，突然失去平衡，倒在地上

工程师在界面里输入一句话：

"为什么这次会撞？"

或者

"为什么这次会失去平衡？"

几秒后，系统给出这样的回答：

事实：碰撞前 80 毫秒，感知模块对该行人输出的置信度仅 0.38，低于过滤阈值 0.60。

原因：该目标在 `object_filter.cpp` 第 89 行被直接过滤丢弃，规划模块未收到目标信息，因此没有刹车。

建议：感知组排查置信度偏低原因（传感器遮挡或光线），或评估调整过滤阈值。

或者，系统给出这样的回答：

事实：起跳后 120ms，IMU 检测到质心横向偏移 4.7cm，超出安全阈值 2.0cm；左踝力矩传感器输出为 0，落地瞬间未收到任何力控指令。

原因：左足底压力传感器在落地前 30ms 出现 18ms 丢包，落地事件未被识别，`whole_body_controller.cpp` 第 312 行的关节状态机未能从"摆动相"切换回"支撑相"，力控持续缺失，质心偏移超出可恢复范围（> 3.5cm），机器人倒地。

建议：硬件组排查左足底传感器丢包原因；控制组补充超时回退逻辑——传感器中断超过 10ms 时自动触发默认落地切换，避免单点故障传导至关节状态机。

每句结论旁边都有证据角标，点开就能看到当时那一帧的数据、具体代码行、参数定义。**所有结论都有据可查。**

同一个事件，不同角色看到的不一样

- **开发视角：**完整的代码路径、参数定义、消息时序，精确到文件行号。
- **测试视角：**代码细节隐藏，只看模块级归因（感知 / 规划 / 控制），直接给出"建议分配给哪个组"，一键导出 Bug 报告。

- **运营视角：**只剩一句自然语言，"系统因目标置信度不足未制动，建议升级给测试团队"，并自动把摘要附进工单。

这个"三层视角"不是为了好看——在真实企业里，外包测试团队需要高质量的 Bug 报告，但核心代码不能对外暴露，这正是今天 Codex / Cursor 类工具做不到的地方。

二、课题核心：做什么、怎么做

本质上是把三件事缝在一起

1. **运行时自动数据采集：**Agent 订阅机器人或自驾系统的关键消息，缓冲异常前后 5 秒的数据，自动触发打包推送。
2. **代码提前静态分析：**提前用 AST 解析器扫一遍源代码，建立"消息名 → 文件行号 → 关联参数"的索引。事件发生时，毫秒级反查，不让 LLM 现场 grep 代码（既慢又容易出错）。
3. **垂直领域的更强大的运行时 Codex：**Codex / Cursor 只读代码，告诉你"这段逻辑可能怎么跑"。Tracero 在此基础上，把代码、参数配置、运行时数据三件事对齐——LLM 拿到的不是源码，而是一个严格组装好的证据包：这一帧数据是谁发的、对应哪个文件哪一行、受哪个参数控制。每条结论必须绑定证据 ID，没有来源的输出直接被系统拒绝。结果是：问"为什么这次倒了"，得到的不是模型的猜测，而是精确到毫秒、行号、参数值的可查证答案。

先从自动驾驶领域开始，做出 MVP，后续再扩展到机器人领域

技术上真正有创新的地方

1. **把"幻觉"尽可能关进笼子** 现在大多数 LLM 应用是把所有数据塞给模型让它自由发挥，幻觉概率很大。Tracero 的路线相反：LLM 没有证据就不能说话，Verifier 自动拒绝任何无来源的输出。
2. **运行时数据和静态代码双层融合** Codex 类工具只看代码，rosvbag 只看数据，两者分开都只能解释一半。Tracero 第一次把"这次实际跑了什么"和"代码里这件事是怎么写的"对齐起来回答问题。
3. **Agent 按需取证** 系统先给 LLM 一个初始证据包，让它自己判断够不够。不够时通过 MCP 工具调用去按需补充——查消息映射、查代码、查运行数据。模型像真正的"工程师助理"一样工作，而不是一次性吞下所有数据。
4. **权限分层呈现** 后端只生成一份完整推理，但每条证据条目都带权限字段，前端按角色实时过滤渲染。同一份数据，三种人看到三种深度。

三、商业化路径

机器人和自动驾驶的异常分析今天仍是"拷数据、围着看一周"。同时，新一轮安全认证标准（ISO PAS 8800、UL 4600 等）已把"运行时证据"列为硬性要求，100 个测试场景的认证材料通常要几个工程师忙几周。Tracero 验证的就是这条路线。

原型阶段结束后有三条孵化方向：

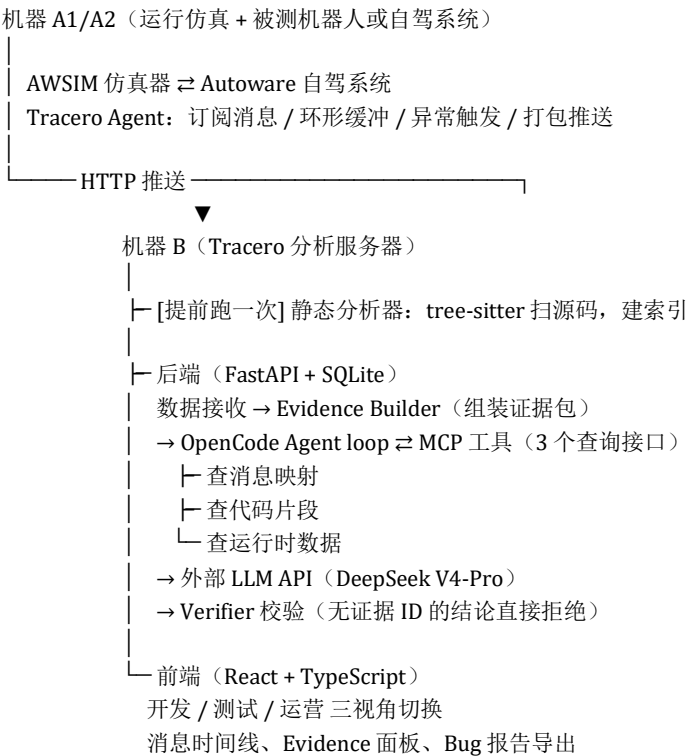
- 1. 面向自驾或机器人公司：提升日常测试和问题收敛效率，按 license 收费。
- 2. 面向有安全认证需求的公司：证据链直接对接 ISO 26262 / PAS 8800 / UL 4600 等认证字段，做增量功能。
- 3. 面向测试工具服务企业：传统工具负责"发现问题"，Tracero 负责"解释问题"，天然互补。

四、原型硬件需求

- 跑自驾系统 + 仿真器的电脑：各需一台有消费级独显的机器
- 跑 Tracero 的服务器：普通 CPU + 内存即可
- 开发 + 演示用电脑一台
- 外部 LLM API 额度（DeepSeek V4-Pro 或同类接口）

预计总费用控制在 5 万元以内。

五、系统结构



六、组队成员分工参考

模块 1：工具类和仿真侧（Tracero Agent +静态分析器） Tracero Agent： 订阅消息、5 秒环形缓冲、规则触发器（急刹检测、感知空窗检测）。运行在自驾或机器人系统侧

静态分析器：提前准备所有消息的流转调用，记录文件名和行号。细致的的字符串 / AST 匹配工作

模块 2：后端 + LLM Agent

最大限度复用既有的前人成果

FastAPI、SQLite、OpenCode 都有成熟文档。真正需要打磨的是 Evidence Builder 和 Verifier 的整合逻辑——怎么组装证据包喂给 OpenCode、怎么校验 LLM 输出。

模块 3：前端 React + TypeScript 生态成熟。

通过前端，把"角色切换"这个演示效果做出彩

七、做完这个项目你能带走什么

ROS 和机器人系统工程：消息订阅、DDS 通信、Autoware 工业级系统的模块结构。这是具身智能和自驾赛道的通用语言，对就业有直接帮助。

更加强大的 codex 开发经历：自己亲手做一个更加强大的，能结合数据理解代码的 codex/Cursor

LLM 应用的工程实践：怎么写一个不胡说的 LLM 应用——幻觉控制、证据驱动、MCP 工具调用。

全栈系统设计：从数据采集 → 后端处理 → 数据库 → 前端可视化的完整链路。队列解耦、异步处理、接口设计——这是工程师最基础也最核心的能力。

产品视角：从"做一个能跑的 Demo"到"做一个三种人都能用的产品"，这个思维转变在课堂里很难有机会真正练到。

八、可复用的开源工具和增量的开发

工具	用途
ROS 2 Humble	机器人通信中间件
Autoware Universe	被测自驾系统（社区文档完整）
AWSIM	开源 Unity 仿真器，直接下 release 包
tree-sitter + tree-sitter-cpp	C++ 代码 AST 解析
FastAPI + SQLite	后端框架
OpenCode + MCP SDK	LLM Agent 执行框架和工具调用协议
DeepSeek V4-Pro API	外部 LLM，OpenAI 兼容接口
React + TypeScript + Vite	前端

大部分模块不是从零发明，而是**组装 + 增量创新**。深入开发部分集中在：Agent、Evidence Builder + Verifier、MCP 工具扩展、前端三视角渲染。

